PROGRAM TUGAS BESAR PEMROGRAMAN II - PBO .NET (C#)
SEMESTER GENAP TAHUN AKADEMIK 2025/2026

LAPORAN PROGRESS TUGAS BESAR: IMPLEMENTASI 4 PILAR OOP PADA SISTEM MOTOCARE

Sistem Informasi Manajemen Bengkel Motor Berbasis C# ASP.NET Core MVC & MySQL

No	Nama Anggota	Peran / Tanggung Jawab Modul	Fokus File / Namespace
1	**Izat Ibrahimovic**	Ketua & Lead Modul 1: Master Data (Pelanggan, Kendaraan, Mekanik)	Models/Modul1_MasterData/ (Person, Customer, Mechanic, Vehicle)
2	**Putrana Yudhistira**	Lead Modul 2: Operasional Servis & Inventaris Gudang Suku Cadang	Models/Modul2_OperasionalServis/ (IServiceItem, BaseServiceItem, Sparepart, ServicePackage, ServiceOrder)
3	**Rafli Apriliandi**	Lead Modul 3: Transaksi Pembayaran, Invoicing, & Generic Repository	Models/Modul3_TransaksiDanRepository/ (IBillable, IRepository<T>, InMemoryRepository<T>, Invoice, ServicePricingCatalog)

■ **KETERANGAN PENGUMPULAN EDLINK:**
Laporan ini disusun khusus untuk memenuhi kriteria evaluasi progress tugas besar pada platform Edlink, yang mewajibkan pembuktian potongan kode: **(1) Variable, get, set, method**; **(2) Inheritance**; **(3) Interface & Abstract**; **(4) Collection, Dictionary, Generic**. Arsitektur kode dirancang modular per folder untuk memastikan pengerjaan mandiri tanpa bentrok file (Zero Merge Conflict).

BAB I: PENDAHULUAN & ARSITEKTUR SISTEM MOTOCARE

MotoCare adalah sistem informasi manajemen bengkel motor berbasis web yang dikembangkan menggunakan **C# ASP.NET Core MVC** dan **MySQL** sebagai proyek tugas besar Pemrograman Berorientasi Objek (PBO) II. Aplikasi ini mengelola alur operasional bengkel secara menyeluruh: registrasi pelanggan & motor, pembagian tugas ke mekanik, penerbitan Surat Perintah Kerja (SPK), manajemen stok suku cadang, hingga faktur tagihan dan pembayaran kasir.

Strategi Pembagian Tugas Mandiri (Anti-Bentrok):
Untuk mencegah terjadinya *conflict* saat pengerjaan tubes, kode program dipisahkan secara tegas ke dalam 3 modul independen berdasarkan tanggung jawab anggota. Setiap anggota mengelola folder dan file kelasnya sendiri dan hanya berkomunikasi melalui Interface atau Base Class yang telah disepakati bersama:

Anggota	Modul & Folder	Tanggung Jawab File	Strategi Anti-Bentrok
Izat Ibrahimovic	Modul 1: Master Data *Models/Modul1_MasterData/*	Person.cs, Customer.cs, Mechanic.cs, Vehicle.cs	Bekerja mandiri pada entitas pengguna. Objek Customer dan Mechanic siap dikonsumsi modul lain.
Putrana Yudhistira	Modul 2: Operasional *Models/Modul2_OperasionalServis/*	IServiceItem.cs, BaseServiceItem.cs, Sparepart.cs, ServicePackage.cs, ServiceOrder.cs	Bekerja mandiri pada logika pengerjaan servis & stok. Mengambil referensi dari Modul 1 tanpa mengubah file Modul 1.
Rafli Apriliandi	Modul 3: Transaksi & Repo *Models/Modul3_TransaksiDanRepository/*	IBillable.cs, IRepository.cs, InMemoryRepository.cs, Invoice.cs, ServicePricingCatalog.cs	Bekerja mandiri pada penagihan kasir & repository. Menggunakan ServiceOrder dari Modul 2 untuk membuat Invoice.

BAB II: PEMETAAN 4 SYARAT UTAMA EVALUASI EDLINK

Berikut adalah matriks pemetaan 4 kriteria wajib yang disyaratkan di form Edlink beserta letak pembuktian kodenya:

Kriteria Edlink	Elemen Teknis C#	Lokasi File	Kegunaan dalam MotoCare

1. Variable, Get, Set, Method	Private fields (_fullName, _unitPrice, _stockQuantity), Auto-properties {get; set;}, Private set, Validasi setter, Business methods.	BaseEntity.cs, Person.cs, Vehicle.cs, Sparepart.cs, ServiceOrder.cs, Invoice.cs	Enkapsulasi & Validasi: Mencegah nilai negatif pada harga/stok, validasi plat nomor motor, serta method bisnis hitung subtotal & kurangi stok.
2. Inheritance	Class inheritance (:) dengan base constructor base(...), method overriding (virtual -> override).	Person -> Customer, Mechanic; BaseEntity -> Semua Entitas; BaseServiceItem -> Sparepart, ServicePackage	Reusability & Hierarki: Atribut umum individu diwariskan ke Customer dan Mechanic tanpa perulangan deklarasi kode.
3. Interface, Abstract	Interface: IServiceItem, IBillable, IRepository<T>. Abstract Class: BaseEntity, BaseServiceItem. Abstract Method: CalculateDiscount().	Models/.../Interfaces/ Models/.../Abstracts/	Kontrak Standar & Abstraksi: Menstandarkan format transaksi kasir (IBillable) dan mewajibkan kelas anak menghitung diskon sesuai jenis item.
4. Collection, Dictionary, Generic	Collection: List<T>, IEnumerable<T>. Dictionary: Dictionary<TKey, TValue>. Generic: IRepository<T>, InMemoryRepository<T>, LookupCatalog<TResult>.	Customer.cs (List<Vehicle>), ServiceOrder.cs (List<BaseServiceItem>), InMemoryRepository.cs, Invoice.cs (Dictionary breakdown), ServicePricingCatalog.cs	Struktur Data Efisien: Menyimpan daftar rincian barang, lookup cepat O(1) berdasarkan ID atau kode servis, serta generic CRUD repository reusable.

BAB III: POTONGAN KODE PROGRAM & PENJELASAN RINCI

3.1 Modul 1: Manajemen Master Data (Penanggung Jawab: Izat Ibrahimovic)

A. File: Person.cs (Base Class, Variable, Get-Set, Method, Virtual Method)

```
// Models/Modul1_MasterData/Person.cs
public class Person : BaseEntity
{
    // Field privat (Enkapsulasi Variable)
    private string _fullName = string.Empty;
    private string _phoneNumber = string.Empty;

    // Get-Set Property dengan validasi logika
    public string FullName
    {
        get => _fullName;
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Nama lengkap tidak boleh kosong.");
            _fullName = value.Trim();
        }
    }

    public string PhoneNumber
    {
        get => _phoneNumber;
        set
        {
            if (string.IsNullOrWhiteSpace(value) || value.Length < 9)
                throw new ArgumentException("Nomor telepon minimal harus 9 digit.");
            _phoneNumber = value.Trim();
        }
    }

    // Method untuk memperbarui kontak
    public void UpdateContact(string newPhone, string newEmail)
    {
        PhoneNumber = newPhone;
        Email = newEmail;
        UpdatedAt = DateTime.Now;
    }

    // Virtual Method (Dapat di-override oleh kelas turunan)
    public virtual string GetProfileSummary()
    {
        return $"[ID: {Id}] {FullName} | Telp: {PhoneNumber}";
    }
}
```

Penjelasan Kode & Kegunaan:

• _fullName & _phoneNumber: Field private untuk menyimpan data anggota dengan proteksi enkapsulasi.

• FullName & PhoneNumber get/set: Memvalidasi data masukan agar nama tidak kosong dan telepon minimal 9 digit.

• UpdateContact(): Method bisnis untuk mengubah nomor telepon/email sekaligus meng-update timestamp UpdatedAt.

• GetProfileSummary(): Virtual method yang dirancang untuk di-override oleh Customer dan Mechanic (Polymorphism).

B. File: Customer.cs (Inheritance dari Person & Collection List)

```
// Models/Modul1_MasterData/Customer.cs
public class Customer : Person
{
    private int _loyaltyPoints;
    public string CustomerCode { get; set; } = string.Empty;

    public int LoyaltyPoints
    {
        get => _loyaltyPoints;
        private set => _loyaltyPoints = value < 0 ? 0 : value;
    }

    // COLLECTION: Daftar motor milik pelanggan (One-to-Many)
    public List<Vehicle> RegisteredVehicles { get; set; } = new List<Vehicle>();

    public Customer(int id, string fullName, string phone, string email, string code, string addr)
        : base(id, fullName, phone, email) // Memanggil konstruktor Base Class
    {
        CustomerCode = code;
        LoyaltyPoints = 0;
    }

    // Method menambah motor ke dalam koleksi List
    public void AddVehicle(Vehicle vehicle)
    {
        if (vehicle == null) throw new ArgumentNullException(nameof(vehicle));
        RegisteredVehicles.Add(vehicle);
    }

    // Override virtual method dari kelas Person
    public override string GetProfileSummary()
    {
        return $"[Pelanggan #{CustomerCode}] {FullName} | Total Motor: {RegisteredVehicles.Count}";
    }
}
```

Penjelasan Kode & Kegunaan:

• public class Customer : Person: Membuktikan INHERITANCE di mana Customer mewarisi seluruh data Person.

• : base(...): Memanggil konstruktor base class Person untuk inisialisasi ID, nama, dan telepon secara rapi.

• List<Vehicle> RegisteredVehicles: Membuktikan COLLECTION untuk menyimpan banyak motor milik satu pelanggan.

• private set pada LoyaltyPoints: Melindungi poin dari manipulasi langsung di luar aturan penukaran resmi.

• override GetProfileSummary(): Mengubah output profil agar mencakup kode pelanggan dan jumlah motor.

C. File: Vehicle.cs (Validasi Plat Nomor, Kilometer, dan Cek Servis Rutin)

```
// Models/Modul1_MasterData/Vehicle.cs
public class Vehicle : BaseEntity
{
    private string _licensePlate = string.Empty;
    private int _currentKilometers;

    public string Brand { get; set; } = string.Empty;
    public string Model { get; set; } = string.Empty;

    public string LicensePlate
    {
        get => _licensePlate;
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Plat nomor motor wajib diisi.");
            _licensePlate = value.Trim().ToUpper(); // Standardisasi otomatis huruf kapital
        }
    }

    public int CurrentKilometers
    {
        get => _currentKilometers;
        set
        {
            if (value < 0) throw new ArgumentException("KM tidak boleh negatif.");
            _currentKilometers = value;
        }
    }

    public bool NeedsPeriodicService() => CurrentKilometers >= 2500;
}
```

Penjelasan Kode & Kegunaan:

• LicensePlate setter: Menstandarkan huruf plat motor menjadi kapital (UPPERCASE) secara otomatis.

• NeedsPeriodicService(): Method bisnis otomatis untuk mengecek apakah motor sudah melewati batas kilometer servis.

3.2 Modul 2: Operasional Servis & Inventaris (Penanggung Jawab: Putrana Yudhistira)

A. File: IServiceItem.cs (Interface) & BaseServiceItem.cs (Abstract Class & Method)

```
// Models/Modul2_OperasionalServis/Interfaces/IServiceItem.cs
public interface IServiceItem
{
    string ItemCode { get; set; }
    string ItemName { get; set; }
    decimal UnitPrice { get; set; }
    int Quantity { get; set; }
    decimal CalculateTotalCost();
}

// Models/Modul2_OperasionalServis/Abstracts/BaseServiceItem.cs
public abstract class BaseServiceItem : BaseEntity, IServiceItem
{
    public string ItemCode { get; set; } = string.Empty;
    public string ItemName { get; set; } = string.Empty;
    public decimal UnitPrice { get; set; }
    public int Quantity { get; set; } = 1;

    public virtual decimal CalculateTotalCost() => UnitPrice * Quantity;

    // ABSTRACT METHOD: Formula diskon wajib diimplementasikan berbeda oleh kelas turunan
    public abstract decimal CalculateDiscount(decimal discountPercentage);
}
```

Penjelasan Kode & Kegunaan:

• interface IServiceItem: Kontrak standar bagi seluruh komponen biaya nota servis (jasa mekanik maupun suku cadang).

• abstract class BaseServiceItem: Kelas abstrak yang menjadi fondasi bagi kelas Sparepart dan ServicePackage.

• abstract decimal CalculateDiscount(...): ABSTRACT METHOD yang memaksa kelas anak menentukan formula diskonnya sendiri.

B. File: Sparepart.cs & ServicePackage.cs (Inheritance & Polymorphism)

```
// Models/Modul2_OperasionalServis/Sparepart.cs
public class Sparepart : BaseServiceItem
{
    private int _stockQuantity;
    public string Brand { get; set; } = string.Empty;

    public int StockQuantity
    {
        get => _stockQuantity;
        set => _stockQuantity = value < 0 ? throw new ArgumentException("Stok tidak boleh minus.") : value;
    }

    public void DeductStock(int amountUsed)
    {
        if (amountUsed > StockQuantity)
            throw new InvalidOperationException($"Stok suku cadang '{ItemName}' tidak cukup.");
        StockQuantity -= amountUsed;
    }

    // Override abstract method: Diskon suku cadang fisik dibatasi maksimal 15%
    public override decimal CalculateDiscount(decimal discountPercentage)
    {
        decimal effectiveDiscount = Math.Min(discountPercentage, 15m);
        return CalculateTotalCost() * (effectiveDiscount / 100m);
    }
}

// Models/Modul2_OperasionalServis/ServicePackage.cs
public class ServicePackage : BaseServiceItem
{
    public int EstimatedMinutes { get; set; }

    // Override abstract method: Diskon jasa servis diperbolehkan promo hingga 30%
    public override decimal CalculateDiscount(decimal discountPercentage)
    {
        decimal effectiveDiscount = Math.Min(discountPercentage, 30m);
        return CalculateTotalCost() * (effectiveDiscount / 100m);
    }
}
```

Penjelasan Kode & Kegunaan:

• Sparepart : BaseServiceItem: Membuktikan INHERITANCE dari Abstract Class.

• DeductStock(): Method kontrol inventaris yang memotong stok otomatis dan mencegah stok bernilai negatif.

• override CalculateDiscount(): Membuktikan POLYMORPHISM di mana diskon sparepart dibatasi 15% sedangkan jasa servis hingga 30%.

C. File: ServiceOrder.cs (Collection List)

```
// Models/Modul2_OperasionalServis/ServiceOrder.cs
public class ServiceOrder : BaseEntity
{
    public string OrderNumber { get; set; } = string.Empty;
    public Customer Customer { get; set; } = null!;
    public Vehicle Vehicle { get; set; } = null!;
    public Mechanic AssignedMechanic { get; set; } = null!;

    // COLLECTION: Menampung rincian jasa dan suku cadang secara polimorfik
    public List<BaseServiceItem> ServiceItems { get; set; } = new List<BaseServiceItem>();

    public void AddServiceItem(BaseServiceItem item)
    {
        if (item == null) throw new ArgumentNullException(nameof(item));
        if (item is Sparepart sp) sp.DeductStock(item.Quantity); // Otomatis kurangi stok
        ServiceItems.Add(item);
    }

    public decimal CalculateSubtotal() => ServiceItems.Sum(x => x.CalculateTotalCost());
}
```

Penjelasan Kode & Kegunaan:

• List<BaseServiceItem> ServiceItems: Membuktikan COLLECTION yang menampung gabungan objek Sparepart dan ServicePackage.

• item is Sparepart sp: Type pattern matching untuk langsung memotong stok suku cadang fisik saat dimasukkan ke SPK servis.

3.3 Modul 3: Transaksi & Generic Repository (Penanggung Jawab: Rafli Apriliandi)

A. File: IRepository.cs & InMemoryRepository.cs (Generic Interface, Generic Class & Dictionary)

```
// Models/Modul3_TransaksiDanRepository/Interfaces/IRepository.cs
public interface IRepository<T> where T : BaseEntity // GENERIC INTERFACE
{
    void Add(T entity);
    T? GetById(int id);
    IEnumerable<T> GetAll();
    int Count();
}

// Models/Modul3_TransaksiDanRepository/Repositories/InMemoryRepository.cs
public class InMemoryRepository<T> : IRepository<T> where T : BaseEntity // GENERIC CLASS
{
    // DICTIONARY: Digunakan sebagai penyimpanan cepat O(1) berdasarkan ID
    private readonly Dictionary<int, T> _storage = new Dictionary<int, T>();
    private int _autoIncrementId = 1;

    public void Add(T entity)
    {
        if (entity == null) throw new ArgumentNullException(nameof(entity));
        if (entity.Id <= 0) entity.Id = _autoIncrementId++;
        _storage.Add(entity.Id, entity);
    }

    public T? GetById(int id) => _storage.TryGetValue(id, out T? item) ? item : null;
    public IEnumerable<T> GetAll() => _storage.Values.ToList();
    public int Count() => _storage.Count;
}
```

Penjelasan Kode & Kegunaan:

• public interface IRepository<T> where T : BaseEntity: Membuktikan GENERIC INTERFACE dengan batasan tipe (constraint).

• public class InMemoryRepository<T>: Membuktikan GENERIC CLASS yang dapat dipakai ulang untuk entitas data mana pun.

• Dictionary<int, T> _storage: Membuktikan DICTIONARY untuk pencarian data instan O(1) berdasarkan ID tanpa scan linear.

B. File: Invoice.cs (Interface IBillable & Dictionary Breakdown Biaya)

```
// Models/Modul3_TransaksiDanRepository/Invoice.cs
public class Invoice : BaseEntity, IBillable // INHERITANCE & INTERFACE
{
    public ServiceOrder ServiceOrder { get; set; } = null!;
    public decimal GrandTotal { get; private set; }
    public bool IsPaid { get; private set; }

    // DICTIONARY: Menyimpan breakdown rincian komponen tagihan (Komponen -> Nominal)
    public Dictionary<string, decimal> FeeBreakdown { get; set; } = new Dictionary<string, decimal>();

    public decimal CalculateGrandTotal()
    {
        decimal subtotal = ServiceOrder.CalculateSubtotal();
        decimal ppn = (subtotal - DiscountAmount) * 0.11m; // PPN 11%
        GrandTotal = (subtotal - DiscountAmount) + ppn;

        // Mengisi DICTIONARY breakdown tagihan
        FeeBreakdown.Clear();
        FeeBreakdown["Subtotal Kotor"] = subtotal;
        FeeBreakdown["Potongan Diskon"] = DiscountAmount;
        FeeBreakdown["Pajak PPN (11%)"] = ppn;
        FeeBreakdown["Grand Total Bersih"] = GrandTotal;
        return GrandTotal;
    }

    public bool ProcessPayment(decimal amountPaid, string paymentMethod)
    {
        if (amountPaid < GrandTotal) throw new InvalidOperationException("Uang pembayaran kurang!");
        IsPaid = true;
        ServiceOrder.MarkAsCompleted(); // Update status SPK servis
        return true;
    }
}
```

Penjelasan Kode & Kegunaan:

• class Invoice : BaseEntity, IBillable: Membuktikan implementasi INTERFACE penagihan dan INHERITANCE dari BaseEntity.

• Dictionary<string, decimal> FeeBreakdown: Membuktikan DICTIONARY untuk mencatat rincian biaya (Jasa, Suku Cadang, Diskon, PPN 11%, Grand Total).

• ProcessPayment(): Method validasi kecukupan uang kasir dan penyelesaian order servis.

C. File: ServicePricingCatalog.cs (Dictionary Tarif & Generic Method)

```
// Models/Modul3_TransaksiDanRepository/ServicePricingCatalog.cs
public class ServicePricingCatalog
{
    // DICTIONARY: Kamus tarif acuan servis resmi bengkel MotoCare
    private readonly Dictionary<string, decimal> _catalog = new Dictionary<string, decimal>(StringComparer.OrdinalIg

    public ServicePricingCatalog()
    {
        _catalog.Add("SRV-TUNEUP", 75000m);
        _catalog.Add("SRV-CVT", 60000m);
        _catalog.Add("SRV-GANTI-OLI", 15000m);
    }

    // GENERIC METHOD: Mengubah tarif menjadi format generic TResult yang diinginkan
    public TResult LookupCatalog<TResult>(string serviceCode, Func<decimal, TResult> valueTransformer)
    {
        if (_catalog.TryGetValue(serviceCode, out decimal price))
            return valueTransformer(price);
        throw new KeyNotFoundException($"Kode '{serviceCode}' tidak terdaftar.");
    }
}
```

Penjelasan Kode & Kegunaan:

• Dictionary<string, decimal>: Menyimpan pasangan kode servis dan nominal harga standar bengkel.

• LookupCatalog<TResult>: Membuktikan GENERIC METHOD yang fleksibel mengembalikan format string, angka, atau objek baru.

BAB IV: HASIL DEMONSTRASI & LOG EKSEKUSI PROGRAM

Pengujian fungsional dilakukan dengan menjalankan file Program.cs via CLI 'dotnet run'. Hasil eksekusi membuktikan bahwa seluruh logika bisnis, pemotongan stok, kalkulasi diskon, dictionary breakdown, dan cetak struk pembayaran berhasil berjalan sempurna:

```
==================================================
              BENGKEL MOTOR MOTOCARE
         Sistem Informasi Manajemen Servis
==================================================
No. Invoice  : INV-MC-20260401-001
No. Order SPK: SPK-2026-0001
Waktu Bayar  : 10/11/2026 08:36:12
Pelanggan    : Budi Santoso (CUST-001)
Kendaraan    : Honda Vario 160 CBS [D 4123 ABC]
Mekanik      : Agus Pratama
--------------------------------------------------
Rincian Pekerjaan & Suku Cadang:
 - Paket Tune-Up Injeksi & CVT x1 : Rp 85.000
 - Oli Mesin SPX 2 Full Synthetic 0.8L x1 : Rp 65.000
 - Busi NGK Laser Iridium CPR9EAIX x1 : Rp 110.000
--------------------------------------------------
Ringkasan Tagihan (Dictionary Breakdown):
   Total Biaya Jasa         : Rp 85.000
   Total Biaya Suku Cadang  : Rp 175.000
   Subtotal Kotor           : Rp 260.000
   Potongan Diskon          : Rp 10.000
   Pajak PPN (11%)          : Rp 27.500
   Grand Total Bersih       : Rp 277.500
--------------------------------------------------
Metode Bayar : QRIS
Nominal Uang : Rp 300.000
Kembalian    : Rp 22.500
Status Lunas : LUNAS (SUKSES)
==================================================
```

BAB V: PANDUAN PENGAMBILAN SCREENSHOT UNTUK EDLINK

Bagi masing-masing anggota kelompok yang ingin mengambil tangkapan layar (screenshot) kode di editor (Visual Studio / VS Code), berikut adalah panduan file dan blok baris kode yang harus disorot:

No	Syarat di Edlink	Nama File Kode	Baris yang Wajib Disorot dalam Screenshot
1	**Variable Get Set Method**	Person.cs & Vehicle.cs	Sorot deklarasi field private _fullName, property FullName { get; set; } dengan validasi, dan method UpdateContact().
2	**Inheritance**	Customer.cs & Sparepart.cs	Sorot baris public class Customer : Person, konstruktor : base(...), dan keyword override.
3	**Interface, Abstract**	IServiceItem.cs, BaseServiceItem.cs, & IBillable.cs	Sorot public interface IServiceItem dan public abstract class BaseServiceItem beserta abstract method CalculateDiscount().
4	**Collection, Dictionary, Generic**	Customer.cs, InMemoryRepository.cs, & Invoice.cs	Sorot List<Vehicle>, generic interface IRepository<T>, serta Dictionary<int, T> dan Dictionary<string, decimal>.

Kesimpulan: Dokumen ini telah merangkum seluruh aspek teknis dan akademis yang disyaratkan dalam form progress tugas besar Edlink. File PDF berukuran ringkas (< 1 MB, jauh di bawah batas maksimal 10 MB) dan siap diunggah sebagai bukti kemajuan proyek MotoCare.